

UNIVERSIDAD AUTÓNOMA DE BAJA CALIFORNIA

FACULTAD DE CIENCIAS QUÍMICAS E INGENIERÍA

INGENIERÍA EN SOFTWARE Y TECNOLOGÍAS EMERGENTES



Gestión de la Innovación | 40027

Grupo 381

**PRÁCTICA V. ANÁLISIS DEL IMPACTO DE LA EVOLUCIÓN TECNOLÓGICA
EN LA INGENIERÍA DE SOFTWARE**

Ing. Rosas Murillo Jorge Abdon

Félix Navarro Jesús Ernesto | 1298853

Fecha de realización: 7 de abril de 2026

Fecha de entrega: 14 de abril de 2026

Índice

Índice.....	2
1. Introducción.....	3
2. Metodología de investigación y criterio de fuentes.....	3
3. Desarrollo histórico y análisis socio-técnico.....	3
3.1 De las herramientas primitivas al registro del conocimiento.....	3
3.2 Del algoritmo a la máquina programable.....	4
3.3 Tabulación, burocracia y nacimiento de la computación moderna.....	4
3.4 Electrónica, redes y la transformación del software en infraestructura.....	4
3.5 De la Web a la inteligencia artificial contemporánea.....	5
4. Espacio reservado para insertar la línea de tiempo visual.....	6
5. Análisis de contrafacticos.....	10
5.1 ¿Qué habría ocurrido sin el transistor?.....	10
5.2 ¿Qué habría ocurrido sin ARPANET?.....	10
6. Síntesis de impacto en Ingeniería de Software.....	10
7. Actividades complementarias.....	11
7.1 Reflexión sobre el documental 'Ada Lovelace: The First Computer Programmer'	11
7.2 Hallazgos del recorrido virtual por el Computer History Museum.....	12
7.3 Ensayo corto: ¿Quién sería el 'Alan Turing' del futuro y qué problema resolvería?.....	12
7.4 Desafío de código: serie de Fibonacci simulando lógica de tarjetas perforadas...13	
Algoritmo en pseudocódigo.....	13
Cuadrícula binaria simplificada.....	13
8. Conclusiones generales.....	14
10. Referencias.....	15

1. Introducción

La Ingeniería de Software no surgió de manera aislada; es el resultado de una larga cadena de innovaciones materiales, lógicas y organizacionales que transformaron la forma en que la humanidad representa información, automatiza tareas y coordina conocimiento. Desde las primeras herramientas líticas hasta la inteligencia artificial contemporánea, cada avance técnico respondió a necesidades históricas concretas: supervivencia, registro, cálculo, comunicación, control, productividad o conectividad. Esta práctica analiza ese proceso histórico para comprender el contexto socio-técnico que hizo posible al software como disciplina de ingeniería.

El informe se elaboró con base en la guía de la Práctica V, que solicita integrar investigación, una línea de tiempo visual, análisis de contrafacticos y una conclusión sobre el papel del transistor y ARPANET en la Ingeniería de Software moderna (Universidad Autónoma de Baja California, s. f.).

2. Metodología de investigación y criterio de fuentes

Para cumplir con la rúbrica, se priorizaron fuentes académicas y repositorios históricos de alta confiabilidad: obras de referencia sobre historia de la computación y de la Ingeniería de Software, además de archivos institucionales del Computer History Museum, CERN, Intel, IBM y Bell Labs. La estrategia de selección buscó tres criterios: exactitud cronológica, identificación de autores y claridad para explicar las condiciones sociales, económicas y técnicas que generaron cada avance.

El análisis se organizó en cinco grandes etapas: 1) herramientas y registro, 2) pensamiento algorítmico y mecanización, 3) tabulación y computación teórica, 4) electrónica y redes, y 5) conectividad global e inteligencia artificial. En cada etapa se identificaron beneficiarios directos, restricciones históricas y efectos sobre la complejidad del software. Este enfoque es coherente con la idea de que la Ingeniería de Software se apoya en capas de proceso, métodos y calidad, pero reposa históricamente en la evolución del hardware y de los sistemas de información (Pressman & Maxim, 2020; Sommerville, 2011).

3. Desarrollo histórico y análisis socio-técnico

3.1 De las herramientas primitivas al registro del conocimiento

Las primeras herramientas de piedra representan el inicio de la tecnología como ampliación de la capacidad humana. Aunque estaban lejos del cómputo, introdujeron una lógica fundamental: resolver problemas mediante instrumentos externos. Más tarde, la escritura permitió conservar instrucciones, deudas, inventarios y reglas; en términos informáticos, hizo posible separar la memoria humana del soporte físico. La aparición de dispositivos de cálculo como el ábaco respondió a la creciente complejidad comercial y administrativa de sociedades urbanas. El beneficio directo recayó en

escribas, comerciantes, administradores y Estados, que pudieron registrar, comparar y decidir con mayor precisión.

3.2 Del algoritmo a la máquina programable

La obra de Al-Juarismi consolidó la noción de procedimiento paso a paso y dio origen al término algoritmo, una idea central para cualquier disciplina de programación. Siglos después, la pascalina y la máquina de Leibniz trasladaron operaciones matemáticas a mecanismos físicos, mientras que el telar de Jacquard mostró que una secuencia de instrucciones podía almacenarse externamente en tarjetas perforadas. En el siglo XIX, Charles Babbage concibió la Máquina Analítica y Ada Lovelace comprendió que una máquina de propósito general no solo podía calcular números, sino manipular símbolos de acuerdo con reglas. Esa intuición anticipó el principio esencial del software: una misma máquina puede ejecutar distintas tareas según el programa que reciba (Isaacson, 2014; Computer History Museum, n.d.-a).

3.3 Tabulación, burocracia y nacimiento de la computación moderna

A finales del siglo XIX, Herman Hollerith usó tarjetas perforadas para acelerar el censo estadounidense de 1890. Este salto respondió a una necesidad administrativa concreta: el Estado y las grandes organizaciones ya no podían procesar manualmente el volumen de datos generado por la sociedad industrial. Con Hollerith aparece una constante de la informática moderna: cuando crece la información, crece la necesidad de automatizar su tratamiento. Posteriormente, Alan Turing formalizó la idea de una máquina universal capaz de ejecutar cualquier procedimiento computable, mientras que ENIAC demostró la potencia del cálculo electrónico a gran escala. El impulso bélico, científico y gubernamental fue decisivo en esta etapa (Ceruzzi, 2003; Computer History Museum, n.d.-b).

3.4 Electrónica, redes y la transformación del software en infraestructura

La invención del transistor en 1947 sustituyó componentes frágiles y voluminosos por dispositivos pequeños, confiables y energéticamente eficientes. Esto permitió reducir costos, aumentar velocidad y abrir el camino a la miniaturización. El circuito integrado y el microprocesador profundizaron esa tendencia, llevando el cómputo desde laboratorios y gobiernos hacia empresas, universidades y hogares. Paralelamente, ARPANET cambió el paradigma de computadoras aisladas a sistemas interconectados; con ello nacieron nuevos problemas de protocolos, interoperabilidad, correo electrónico, acceso remoto y seguridad. La historia del software dejó entonces de ser solo la historia de programas locales y pasó a ser la historia de ecosistemas conectados (Naughton, 2016; Computer History Museum, n.d.-c; Intel, n.d.).

3.5 De la Web a la inteligencia artificial contemporánea

La World Wide Web democratizó el acceso a la información al volver navegable Internet para públicos no especializados. A partir de entonces, el software adquirió escala planetaria: comercio electrónico, colaboración global, servicios en línea y plataformas de desarrollo distribuido. Más recientemente, la inteligencia artificial ha trasladado la atención desde la automatización de reglas hacia la inferencia a partir de datos. Sin embargo, la IA contemporánea no puede entenderse sin los cimientos previos: hardware miniaturizado, redes globales, sistemas operativos, lenguajes, estándares y una disciplina de Ingeniería de Software capaz de gestionar complejidad, confiabilidad y mantenimiento a gran escala (CERN, n.d.; Pressman & Maxim, 202

4. Espacio reservado para insertar la línea de tiempo visual

Inserta en esta sección la imagen final de la línea de tiempo elaborada en la herramienta que prefieras. Se recomienda conservar la misma secuencia cronológica, el uso de imágenes representativas y una codificación por colores para distinguir etapas históricas.



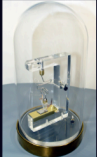
TERCERA REVOLUCIÓN INDUSTRIAL

Línea de Tiempo

Felix Navarro Jesus Ernesto - 1298853

1947 - Invención del transistor

- **Autor(es):** John Bardeen, Walter Brattain y William Shockley
- **Palabra clave:** Miniatrización
- **Impacto:** El transistor sustituyó a las válvulas de vacío y abrió la puerta a dispositivos más pequeños, rápidos y confiables. Es uno de los pilares de toda la revolución digital.



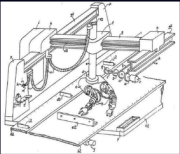
1948 - Presentación pública del transistor de unión

- **Autor(es):** William Shockley
- **Palabra clave:** Escalabilidad
- **Impacto:** La mejora del transistor facilitó su producción y su uso práctico en la industria electrónica, acelerando su adopción comercial.



1954 - Patente del brazo programable que originó Unimate

- **Autor(es):** George Devol
- **Palabra clave:** Automatización
- **Impacto:** Este diseño sentó las bases de la robótica industrial, mostrando que la revolución no solo era digital, sino también productiva.



1956 - Fundación de Unimation

- **Autor(es):** George Devol y Joseph Engelberger
- **Palabra clave:** Robótica
- **Impacto:** Se crea la primera empresa dedicada a robots industriales, marcando el paso de la invención a la comercialización de la automatización.



1958 - Primer circuito integrado

- **Autor(es):** Jack Kilby
- **Palabra clave:** Integración
- **Impacto:** Kilby demostró que varios componentes electrónicos podían incorporarse en un solo chip, reduciendo tamaño y complejidad.





1959 – Circuito integrado práctico en silicio

- **Autor(es):** Robert Noyce
- **Palabra clave:** Producción masiva
- **Impacto:** Noyce perfeccionó el circuito integrado para hacerlo más viable industrialmente, sentando la base de la microelectrónica moderna.

1961 – Unimate entra a la industria automotriz

- **Autor(es):** George Devol y Joseph Engelberger
- **Palabra clave:** Manufactura
- **Impacto:** General Motors instala Unimate en su línea de producción, iniciando la automatización robótica en fábricas.

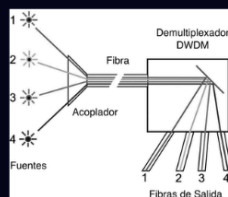
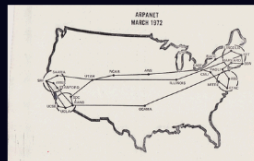


1969 – Nace UNIX en Bell Labs

- **Autor(es):** Ken Thompson y Dennis Ritchie
- **Palabra clave:** Sistema operativo
- **Impacto:** UNIX aportó portabilidad, modularidad y una lógica de desarrollo que influyó profundamente en software, redes e Internet.

1969 – ARPANET conecta sus primeros nodos

- **Autor(es):** ARPA, UCLA, SRI; Charley Kline y Bill Duvall en la primera transmisión
- **Palabra clave:** Conectividad
- **Impacto:** ARPANET fue la base técnica de Internet y transformó la comunicación digital entre computadoras.

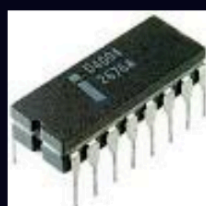


1970 – Fibra óptica de baja pérdida

- **Autor(es):** Robert Maurer, Donald Keck y Peter Schultz
- **Palabra clave:** Transmisión
- **Impacto:** La fibra óptica hizo posible enviar grandes volúmenes de información a alta velocidad y a largas distancias.

1971 – Primer correo electrónico en red

- **Autor(es):** Ray Tomlinson
- **Palabra clave:** Comunicación
- **Impacto:** El e-mail convirtió a las redes en herramientas de interacción cotidiana y popularizó el uso del símbolo @ en direcciones.



1971 – Lanzamiento del Intel 4004

- **Autor(es):** Federico Faggin, Marcian Hoff, Stan Mazor y Masatoshi Shima
- **Palabra clave:** Microprocesador
- **Impacto:** El Intel 4004 fue el primer microprocesador comercial y concentró el poder de cómputo en un solo chip, iniciando la computación moderna.

1973 - Primera llamada desde un teléfono móvil portátil

- **Autor(es):** Martin Cooper, Motorola
- **Palabra clave:** Movilidad
- **Impacto:** Este hecho anticipó la revolución de la telefonía móvil y la comunicación ubicua.



1973 - Propuesta de Ethernet

- **Autor(es):** Robert Metcalfe, con desarrollo posterior junto a David Boggs
- **Palabra clave:** Redes locales
- **Impacto:** Ethernet hizo posible conectar computadoras en oficinas y centros de trabajo, convirtiéndose en estándar para redes locales.

1975 - Altair 8800 y auge de la microcomputación

- **Autor(es):** Ed Roberts
- **Palabra clave:** Democratización
- **Impacto:** El Altair 8800 acercó la computación a aficionados, estudiantes y pequeños desarrolladores, impulsando la informática personal.



1977 - Popularización de la computadora personal

- **Autor(es):** Steve Wozniak y Steve Jobs con Apple II; también Tandy y Commodore en la misma etapa
- **Palabra clave:** Accesibilidad
- **Impacto:** La computadora personal dejó de ser una herramienta exclusiva de laboratorios y grandes empresas.

1981 - Lanzamiento de la IBM PC

- **Autor(es):** IBM, con liderazgo de Don Estridge; basada en Intel 8088
- **Palabra clave:** Estandarización
- **Impacto:** La IBM PC consolidó el mercado de computadoras personales y estableció una arquitectura que dominó la informática empresarial.



1989 - Propuesta de la World Wide Web - Altair 8800 y auge de la microcomputación

- **Autor(es):** Tim Berners-Lee, con formalización posterior junto a Robert Cailliau
- **Palabra clave:** Web
- **Impacto:** La Web transformó Internet en un entorno accesible para compartir información mediante hipertexto y navegadores gráficos a usuarios fuera de grandes instituciones.

1991 - Primer sitio web y despliegue comercial de GSM

- **Autor(es):** Tim Berners-Lee para el primer sitio web; desarrollo europeo coordinado por actores de la industria móvil como Ericsson y Radiolinja para GSM
- **Palabra clave:** Globalización digital
- **Impacto:** En 1991 la Web comenzó a existir públicamente y GSM llevó la telefonía digital móvil al mercado, consolidando la conectividad global.



Figura 1. Línea de tiempo de la evolución tecnológica desde las herramientas primitivas hasta la IA contemporánea.

5. Análisis de contrafácticos

5.1 ¿Qué habría ocurrido sin el transistor?

Sin el transistor, la evolución del hardware habría permanecido anclada a las válvulas de vacío por mucho más tiempo. Las computadoras seguirían siendo grandes, costosas, frágiles y de alto consumo energético. En ese escenario, el software habría continuado como una actividad extremadamente especializada, ligada a gobiernos, laboratorios militares y centros científicos con grandes presupuestos. No habrían surgido con la misma velocidad la informática personal, los dispositivos móviles, la electrónica de consumo ni la miniaturización necesaria para sensores, redes y sistemas embebidos. En términos de Ingeniería de Software, esto habría retrasado el desarrollo de metodologías de producción masiva, pruebas a gran escala, usabilidad orientada al público general y ecosistemas de desarrollo para millones de usuarios. La disciplina existiría, pero sería mucho menos democrática, menos iterativa y menos integrada a la vida cotidiana.

5.2 ¿Qué habría ocurrido sin ARPANET?

Si ARPANET no hubiera existido, la historia del software habría quedado dominada por sistemas locales y aislados durante más tiempo. La falta de una red pionera habría retrasado la experimentación temprana con protocolos, correo electrónico, acceso remoto, distribución de recursos y colaboración entre instituciones. Sin esa base, Internet no habría madurado del mismo modo y la Ingeniería de Software habría tardado más en enfrentar problemas hoy centrales: interoperabilidad, concurrencia distribuida, servicios en red, seguridad, sincronización y escalabilidad global. También se habría demorado la cultura de colaboración abierta que hoy caracteriza a comunidades de desarrollo, repositorios distribuidos, servicios en la nube y despliegues continuos. En síntesis, sin ARPANET el software moderno sería menos conectado, menos colaborativo y menos ubicuo.

6. Síntesis de impacto en Ingeniería de Software

La invención del transistor y la creación de la red ARPANET son dos de los acontecimientos más decisivos para entender por qué la Ingeniería de Software moderna existe en la forma en que la conocemos actualmente. El transistor fue el punto de partida de la miniaturización y la confiabilidad en la electrónica. Al sustituir a las válvulas de vacío, permitió construir computadoras más pequeñas, rápidas, económicas y estables. Sin este avance, el desarrollo del hardware habría seguido siendo demasiado costoso, voluminoso e ineficiente, lo que habría limitado seriamente la evolución del software. En otras palabras, sin transistor no habría existido la base material necesaria para diseñar sistemas computacionales complejos, accesibles y escalables.

Por otra parte, ARPANET transformó el uso de las computadoras al pasar de máquinas aisladas a

sistemas interconectados. Esta red no solo fue el antecedente directo de Internet, sino que también introdujo nuevas necesidades técnicas: comunicación entre sistemas, transferencia de datos, protocolos, seguridad, interoperabilidad y trabajo colaborativo a distancia. Todas estas necesidades impulsaron el surgimiento de nuevas áreas dentro de la Ingeniería de Software, como la programación de redes, los sistemas distribuidos, la arquitectura cliente-servidor, el desarrollo web y, posteriormente, la computación en la nube.

La relación entre ambos avances es fundamental. El transistor hizo posible el crecimiento del poder de cómputo y la masificación de los dispositivos; ARPANET convirtió ese poder de cómputo en una red de comunicación global. Gracias a esta combinación, el software dejó de ser una herramienta limitada a una sola máquina y se convirtió en un elemento central de la vida económica, social, científica e industrial.

Por ello, puede afirmarse que el transistor aportó la base física de la revolución digital, mientras que ARPANET aportó la base de conectividad. Sin ambos, la Ingeniería de Software moderna no habría alcanzado su alcance, complejidad e impacto actual.

7. Actividades complementarias

7.1 Reflexión sobre el documental 'Ada Lovelace: The First Computer Programmer'

La figura de Ada Lovelace resulta especialmente relevante porque permite comprender que la Ingeniería de Software no nació únicamente del hardware, sino también de una forma abstracta de pensar las máquinas. Su aporte más valioso no fue haber construido un dispositivo, sino haber advertido que una máquina programable podía manipular símbolos mediante instrucciones, y no solo números para cálculos aislados. Esa visión es extraordinaria para el siglo XIX porque anticipa la idea moderna de programa, es decir, una secuencia formal de operaciones capaz de ser reutilizada, adaptada y generalizada.

La relación con la inteligencia artificial aparece justamente en ese nivel de abstracción. Cuando Lovelace imagina que una máquina podría operar sobre símbolos musicales u otros tipos de representación, está abriendo la puerta a pensar el cómputo como transformación simbólica y no solo aritmética. Aunque ella misma consideraba que la máquina no 'creaba' por sí sola, su marco conceptual fue indispensable para que generaciones posteriores imaginaran sistemas que analizan lenguaje, imágenes o patrones complejos. En ese sentido, su legado no está solo en un algoritmo histórico, sino en haber imaginado que el futuro del cómputo sería general, programable y potencialmente creativo.

7.2 Hallazgos del recorrido virtual por el Computer History Museum

El recorrido virtual por los archivos del Computer History Museum permite apreciar algo que en los textos a veces se pierde: la materialidad de la historia informática. Ver artefactos como tarjetas perforadas, réplicas del transistor, el ENIAC o los primeros equipos conectados a ARPANET muestra que cada salto tecnológico implicó resolver restricciones físicas muy concretas: tamaño, calor, consumo, costo, capacidad de almacenamiento o velocidad de transmisión. La historia del software no puede separarse de estas limitaciones materiales.

Otro hallazgo importante es la continuidad entre tecnologías que, a primera vista, parecen lejanas. Las tarjetas de Jacquard, la tabulación de Hollerith y el correo electrónico sobre ARPANET pertenecen a épocas distintas, pero comparten una lógica común: codificar información, automatizar su procesamiento y ampliar la coordinación entre personas e instituciones. El museo permite observar que la Ingeniería de Software moderna es heredera de esa continuidad histórica y que su complejidad actual solo fue posible gracias a una secuencia acumulativa de innovaciones, más que a inventos aislados.

7.3 Ensayo corto: ¿Quién sería el 'Alan Turing' del futuro y qué problema resolvería?

Si hoy tuviéramos que imaginar al 'Alan Turing' del futuro, probablemente no pensaríamos en una sola persona aislada, sino en un perfil interdisciplinario capaz de unir teoría, ética, infraestructura y diseño de sistemas. Aun así, el equivalente contemporáneo sería alguien que resolviera uno de los problemas más difíciles del presente: construir inteligencia artificial confiable, verificable y alineada con fines humanos en entornos de alta complejidad social.

Alan Turing no fue grande únicamente por sus aportes matemáticos, sino porque logró formular preguntas fundacionales. Preguntó qué significa computar, qué problemas pueden resolverse mecánicamente y hasta qué punto una máquina puede exhibir comportamiento inteligente. El 'Turing del futuro' tendría una tarea similar, pero aplicada a una nueva escala: no solo preguntaría si un sistema puede generar respuestas convincentes, sino cómo garantizar que dichas respuestas sean correctas, auditables, justas y seguras cuando impactan salud, justicia, educación, seguridad o infraestructura crítica.

Ese futuro pionero o pionera tendría que resolver el problema de la confianza en sistemas autónomos. Hoy ya no basta con que una IA funcione; debe poder explicarse, monitorearse, corregirse y gobernarse. Por eso, el desafío más grande no es crear modelos cada vez más poderosos, sino crear una arquitectura técnica e institucional que combine capacidad con responsabilidad. En términos de Ingeniería de Software, esto implica nuevos métodos de verificación, trazabilidad de datos, pruebas para sesgo, diseño centrado en el riesgo y mecanismos de supervisión humana significativa.

También tendría que trabajar en un entorno radicalmente distinto al de Turing. Mientras que Turing operó en una época donde la pregunta central era si la máquina podía pensar, el problema actual es cómo convivir con sistemas que ya participan en decisiones reales. El nuevo 'Turing' no resolvería solamente un problema lógico, sino un problema civilizatorio: cómo hacer que la inteligencia computacional amplíe capacidades humanas sin degradar la autonomía, la privacidad o la dignidad.

Por ello, el 'Alan Turing' del futuro sería quien lograra establecer los fundamentos científicos y éticos de una IA verdaderamente responsable. Su gran contribución no sería solo demostrar que una máquina puede hacer más, sino garantizar que ese 'más' sea compatible con el bienestar humano y con sistemas de software confiables a escala global.

7.4 Desafío de código: serie de Fibonacci simulando lógica de tarjetas perforadas

A continuación se presenta una versión simple del algoritmo de Fibonacci y una representación didáctica inspirada en tarjetas perforadas. No reproduce el estándar histórico exacto de una tarjeta IBM, sino una cuadrícula binaria simplificada para mostrar cómo una instrucción puede codificarse mediante presencia o ausencia de perforaciones.

Algoritmo en pseudocódigo

```

Inicio
  n <- 7
  a <- 0
  b <- 1
  Repetir mientras n > 0
    Escribir a
    temp <- a + b
    a <- b
    b <- temp
    n <- n - 1
  FinRepetir
Fin

```

Cuadrícula binaria simplificada

Tarjeta	Instrucción	b7	b6	b5	b4	b3	b2	b1	b0
T1	CARGAR n=7	0	0	0	0	0	1	1	1

Tarjeta	Instrucción	b7	b6	b5	b4	b3	b2	b1	b0
T2	CARGAR a=0	0	0	0	1	0	0	0	0
T3	CARGAR b=1	0	0	0	1	0	0	0	1
T4	IMPRIMIR a	0	1	0	0	0	0	0	1
T5	SUMAR a+b -> temp	0	0	1	0	0	0	1	1
T6	MOVER b -> a	0	0	1	1	0	0	1	0
T7	MOVER temp -> b	0	0	1	1	0	1	0	0
T8	DECREMENTAR n	0	1	0	1	0	0	0	0
T9	SI n>0, VOLVER T4	1	0	0	0	0	1	0	1

Interpretación: un '1' representa una perforación y un '0' la ausencia de perforación. La secuencia de tarjetas describe carga de valores, salida, suma, actualización de variables y salto condicional. Esta representación permite visualizar que el software, incluso en sus formas más tempranas, depende de instrucciones discretas, ordenadas y codificadas.

8. Conclusiones generales

El recorrido histórico confirma que la Ingeniería de Software es inseparable de la evolución tecnológica de largo plazo. La humanidad primero desarrolló herramientas, luego sistemas de registro, después modelos formales para describir procedimientos y finalmente máquinas capaces de ejecutar esos procedimientos con velocidad creciente. Cada etapa resolvió un problema concreto y, al mismo

tiempo, creó nuevas complejidades que más tarde demandaron mejores formas de diseñar, verificar y mantener software.

El transistor y ARPANET sobresalen porque transformaron las dos dimensiones más determinantes del software moderno: capacidad de cómputo y conectividad. A partir de ellas, el software dejó de ser local, raro y costoso para convertirse en infraestructura global. Por ello, entender la historia tecnológica no es un ejercicio decorativo, sino una condición para comprender por qué hoy la Ingeniería de Software debe ocuparse de escalabilidad, seguridad, ética, mantenimiento, interoperabilidad y calidad de servicio.

10. Referencias

- Brooks, F. P. (1995). *The Mythical Man-Month: Essays on Software Engineering* (Anniversary ed.). Addison-Wesley.
- CERN. (n.d.). The birth of the Web. <https://home.cern/science/computing/birth-web>
- Ceruzzi, P. E. (2003). *A History of Modern Computing* (2nd ed.). MIT Press.
- Computer History Museum. (n.d.-a). Ada Lovelace | Babbage Engine.
<https://www.computerhistory.org/babbage/adalovelace/>
- Computer History Museum. (n.d.-b). ENIAC.
<https://www.computerhistory.org/revolution/birth-of-the-computer/4/78>
- Computer History Museum. (n.d.-c). Internet history: 1970s.
<https://www.computerhistory.org/internethistory/1970s/>
- Computer History Museum. (n.d.-d). Punched cards control Jacquard loom.
<https://www.computerhistory.org/storageengine/punched-cards-control-jacquard-loom/>
- Computer History Museum. (n.d.-e). 1947: Invention of the point-contact transistor.
<https://www.computerhistory.org/siliconengine/invention-of-the-point-contact-transistor/>
- Intel. (n.d.). Announcing a new era of integrated electronics.
<https://www.intel.com/content/www/us/en/history/virtual-vault/articles/the-intel-4004.html>
- Isaacson, W. (2014). *The Innovators: How a Group of Hackers, Geniuses, and Geeks Created the Digital Revolution*. Simon & Schuster.
- IBM. (n.d.). The IBM Personal Computer. <https://www.ibm.com/history/personal-computer>
- Naughton, J. (2016). *A Brief History of the Future: Origins of the Internet*. Hachette UK.
- Nokia Bell Labs. (n.d.). 50 years of Unix.
<https://www.nokia.com/bell-labs/about/history/innovation-stories/50-years-unix/>
- Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education.
- Sommerville, I. (2011). *Software Engineering* (9th ed.). Pearson.

Tanenbaum, A. S., & Wetherall, D. J. (2011). Computer Networks (5th ed.). Pearson.

Universidad Autónoma de Baja California. (s. f.). Práctica V: Análisis del impacto de la evolución tecnológica en la Ingeniería de Software. Documento de clase.